	Mode require careful implementation.	
Asynchronous state updates	Handling asynchronous operations (e.g., API calls) alongside state updates can lead to complex, hard-to-manage code when using native tools.	Middleware support in libraries like Redux Thunk or Redux Saga simplifies handling side effects such as API calls.
Performance bottlenecks	React's Context API can lead to unnecessary re-renders if not optimized correctly, especially in large applications.	Libraries like Zustand, Jotai, or Recoil optimize re-renders by tracking which components depend on specific parts of the state.

4.3 Built-in hooks and custom hooks

React Hooks are a frequent focus area in technical interviews. Introduced in React 16.8, Hooks transformed how developers manage state and lifecycle behavior within functional components. They allow function components to handle state (`useState`), perform side effects (`useEffect`), and work with mutable references (`useRef`)—all without relying on class components. This paradigm shift simplifies React development by promoting modular, reusable, and maintainable code.

Table 4.7 highlights essential aspects of React Hooks to help you prepare for common interview questions.

Table 4.7 React Hooks

Topic	Fundamentals	Applied skills	Advanced
Builtin Hooks	Core Hooks, like `useState`, `useEffect`, etc.	Use hooks for state and side effects. Compose hooks for	Optimize re-renders, avoid stale closures, integrate